RFI Response — Strategic Development Partnership for Intelli-Q

Submitted by Altysys

Date: 22 February 2026
Prepared for: Nomiso
Document Classification: Confidential

Table of Contents

- 1. Executive Summary
- 2. Architecture, Backend Development & Integration Approach
  - 2.1 Architectural Philosophy
  - 2.2 LLM Orchestration & RAG Architecture
  - 2.3 Backend Engineering Approach
  - 2.4 Integration Strategy
  - 2.5 Security & Governance
- 3. UI/UX & Product Engineering Maturity
  - 3.1 Design Philosophy
  - 3.2 UI Engineering Approach
  - 3.3 Product Engineering Discipline
- 4. Module Breakdown & Effort Estimation
  - 4.1 Section A — UI/UX & Workflow Layer Enhancements
  - 4.2 Section B — Engineering Intelligence Modules
  - 4.3 Cross-Cutting Concerns
  - 4.4 Effort Summary
- 5. Proposed Engagement Model & Team Structure
  - 5.1 Engagement Model
  - 5.2 Proposed Team Composition
  - 5.3 Delivery Methodology
  - 5.4 Phased Timeline
  - 5.5 Risk Mitigation
  - 5.6 IP Protection & Data Privacy
- 6. Why Altysys

---

1. Executive Summary

Altysys is pleased to submit this response to Nomiso's Request for Information for the strategic development partnership on **Intelli-Q**, the AI-powered SDLC & Engineering Intelligence Platform.

We bring deep, production-proven expertise in building enterprise-grade AI platforms on Azure, microservices architectures, LLM orchestration, and product engineering — evidenced by our portfolio spanning IoT fleet intelligence platforms, enterprise search & knowledge systems, and full-stack product builds across regulated and high-scale domains.

Our approach to Intelli-Q is rooted in three principles:

1. **Extend, don't replace** — We will build on top of Intelli-Q's existing architecture (RAG knowledge base, JIRA integrations, test generation engines) rather than introducing parallel systems.
2. **AI-native engineering** — Every new module is designed around agentic workflows, structured validation layers, and human-in-the-loop governance — not AI bolted onto traditional CRUD.
3. **Enterprise-grade from day one** — RBAC, audit logging, telemetry, scalable data pipelines, and security controls are embedded in the architecture, not retrofitted.

We are positioned to be a long-term co-innovation partner, not a staff augmentation vendor. Our proposal reflects an opinionated architecture perspective, a clear execution plan, and a team structure designed for sustained velocity across the 11 modules in scope.

---

2. Architecture, Backend Development & Integration Approach

2.1 Architectural Philosophy

We propose evolving Intelli-Q on a **modular, event-driven microservices architecture** deployed on Azure, designed for:

Principle	Implementation
Modularity	Each capability (SBP Generation, Code Review, Impact Analysis, etc.) is an independently deployable service with its own data boundary
Extensibility	New AI agents and modules plug into a shared orchestration layer without disrupting existing capabilities
Observability	Structured logging, distributed tracing (Application Insights), and AI-specific telemetry (prompt latency, token usage, hallucination scores)
Resilience	Circuit breakers, retry policies, dead-letter queues for async AI workloads; graceful degradation when LLM services are unavailable

High-Level Architecture Layers:

```
┌──────────────────────────────────────────────────────────────────┐
│                        PRESENTATION LAYER                        │
│   React SPA  ·  VS Code Extension  ·  JIRA Plugin  ·  API Portal  │
├──────────────────────────────────────────────────────────────────┤
│                     API GATEWAY / BFF LAYER                      │
│   Azure API Management  ·  Authentication  ·  Rate Limiting      │
```

```
├──────────────────────────────────────────────────────────────────────────┤
│                    ORCHESTRATION & AGENT LAYER                           │
│   Agentic Workflow Engine  ·  LLM Router  ·  Validation Pipeline          │
│   Task Queue (Service Bus)  ·  Human-in-the-Loop Gateway                  │
├──────────────────────────────────────────────────────────────────────────┤
│                      DOMAIN SERVICE LAYER                                 │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   │ SBP Gen  │  │ HLD/LLD  │  │ Code Gen │  │ Code Rev │  │ Impact   │  │
│   │ Service  │  │ Service  │  │ Service  │  │ Service  │  │ Analysis │  │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
│   ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────┐  │
│   │ Unit Test│  │ Script   │  │ Framework│  │ Design   │  │ Code     │  │
│   │ Gen Svc  │  │ Gen Svc  │  │ Migration│  │ Assist   │  │ Assess   │  │
│   └──────────┘  └──────────┘  └──────────┘  └──────────┘  └──────────┘  │
├──────────────────────────────────────────────────────────────────────────┤
│                       DATA & KNOWLEDGE LAYER                              │
│   Vector DB (RAG)  ·  Azure SQL  ·  Blob Storage  ·  Redis Cache          │
│   Cosmos DB (event store)  ·  Azure Cognitive Search                      │
├──────────────────────────────────────────────────────────────────────────┤
│                      INTEGRATION & INFRA LAYER                            │
│   JIRA  ·  Git Repos  ·  SonarQube  ·  CI/CD  ·  External Tools           │
│   Azure AD  ·  Key Vault  ·  Front Door  ·  App Insights                  │
└──────────────────────────────────────────────────────────────────────────┘
```

2.2 LLM Orchestration & RAG Architecture

This is the heart of Intelli-Q. Our approach:

Agentic Workflow Engine

- Multi-agent orchestration using a **directed acyclic graph (DAG)** execution model
- Each module registers its AI agents (e.g., `CodeReviewAgent`, `HLDGeneratorAgent`, `ImpactAnalysisAgent`) with the engine
- Agents can chain, branch, and converge — enabling complex multi-step workflows (e.g., "Analyze code → Generate HLD → Validate against requirements → Produce traceability report")
- Built-in support for **parallel agent execution** where sub-tasks are independent

LLM Router

- Abstraction layer over multiple LLM providers (Azure OpenAI, Anthropic, open-source models)
- Model selection based on task type, latency requirements, cost thresholds, and security classification
- Fallback chains for resilience (primary → secondary → cached response)
- Token budget management and cost attribution per module/tenant

RAG Pipeline Enhancement

- Extend the existing Knowledge Base with **hierarchical chunking** — document-level, section-level, and paragraph-level embeddings for variable-granularity retrieval
- **Hybrid search** combining vector similarity + keyword search (BM25) via Azure Cognitive Search

- **Contextual re-ranking** using cross-encoder models to improve retrieval precision
- **Knowledge graph overlay** — extract entity relationships from ingested documents to enable graph-based reasoning alongside vector retrieval
- **Freshness scoring** — weight more recent documents higher for rapidly evolving codebases

Structured Validation Layer

- Every AI-generated output passes through a validation pipeline before reaching the user:
  1. **Schema validation** — Output conforms to expected structure (e.g., HLD must have component diagram data, dependency list, etc.)
  2. **Grounding check** — Cross-reference generated content against RAG context; flag ungrounded claims
  3. **Policy compliance** — Enforce enterprise rules (naming conventions, security patterns, coding standards)
  4. **Confidence scoring** — Surface confidence levels to users; auto-route low-confidence outputs to human review

2.3 Backend Engineering Approach

API Design

- RESTful APIs for synchronous operations (CRUD, configuration, queries)
- WebSocket / Server-Sent Events for streaming AI generation outputs to the UI in real-time
- Async job APIs for long-running operations (code analysis, large-scale test generation) with job status polling and webhook notifications
- OpenAPI 3.1 specs auto-generated from code, published to developer portal

Data Architecture

- **Azure SQL** for relational domain data (projects, users, configurations, traceability links)
- **Cosmos DB** for event sourcing and audit trail (every AI generation, edit, approval captured as immutable events)
- **Vector Store** for RAG embeddings (extending existing Knowledge Base)
- **Azure Blob** for artifact storage (generated documents, code files, diagrams)
- **Redis** for caching frequently accessed context, session management, and rate limiting

Service Communication

- Synchronous: gRPC for internal service-to-service calls (low-latency, type-safe)
- Asynchronous: Azure Service Bus for event-driven workflows (job queuing, notification fans, cross-module triggers)
- CQRS pattern where appropriate — separate read/write models for high-throughput modules like Code Impact Analysis

2.4 Integration Strategy

We approach integration with a "meet enterprises where they are" philosophy:

Integration Point	Approach
JIRA	Extend existing integration — bi-directional sync for requirements, test cases, and now design artifacts (HLD/LLD linked to epics/stories)
Git Repositories	GitHub / Azure DevOps / GitLab connectors for code ingestion, PR-triggered code review, and generated code push-back
CI/CD Pipelines	Webhook-based triggers for automated test execution, code quality gates, and deployment risk scoring
Static Analysis Tools	API integration with SonarQube, Veracode, Checkmarx — aggregate findings into Code Assessment dashboard
IDE Extensions	Extend existing VS Code extension for inline code suggestions, impact analysis previews, and design assistance
Export Formats	PDF, DOCX, Confluence, and structured API responses for all generated artifacts

2.5 Security & Governance

Concern	Implementation
Authentication	Azure AD / OIDC integration; SSO support
Authorization	Granular RBAC — module-level, project-level, and action-level permissions
Data Encryption	TLS 1.3 in transit; AES-256 at rest; customer-managed keys via Key Vault
Audit Logging	Immutable event log for every AI generation, human edit, approval, and data access
Prompt Security	Input sanitization, prompt injection detection, PII redaction in logs
Network Security	VNET integration, Private Endpoints, Azure Front Door with WAF
Compliance	SOC 2 Type II alignment; GDPR-aware data handling

---

3. UI/UX & Product Engineering Maturity

3.1 Design Philosophy

Intelli-Q's expansion into 11+ new modules demands a **design system approach**, not page-by-page design. Our philosophy:

- **Progressive disclosure** — Complex AI workflows surfaced through guided, multi-step interfaces. Users see what they need at each stage, not everything at once.
- **AI-transparent UX** — Users always see what the AI is doing, what sources it used, and confidence levels. No black-box generation.
- **Developer-centric ergonomics** — Engineers are the primary users. Interfaces optimized for keyboard navigation, code-centric views, diff comparisons, and syntax highlighting.

- **Consistent vocabulary** — A shared design system ensures that "generation," "review," "approval," and "export" work identically across all modules.

3.2 UI Engineering Approach

Design System

- Component library built on React with a token-based design system (spacing, typography, color, elevation)
- Shared components: AI output viewer (streaming + final), diff viewer, approval workflow panel, traceability sidebar, export controls
- Storybook-driven development for visual regression testing and design-dev alignment

Key UI Patterns Across Modules

Pattern	Used In	Description
Multi-Step Wizard	SBP Generation, HLD/LLD, Code Generation	Guided creation flows with back/forward navigation, step validation, and draft saving
Split-Pane Editor	Unit Test Gen, Script Gen, Code Review	Source on left, AI-generated output on right, with inline editing and diff highlighting
Streaming Output	All generation modules	Real-time token-by-token rendering with progress indicators and cancel capability
Traceability Panel	All modules	Collapsible sidebar showing artifact lineage (Story → Design → Code → Test)
Review & Approve	All modules	Inline commenting, approval workflows, version history, and rollback
Dashboard View	Code Assessment, Impact Analysis	Aggregated metrics, charts, drill-down capability

Accessibility & Performance

- WCAG 2.1 AA compliance
- Code splitting and lazy loading per module to maintain sub-2s initial load
- Optimistic UI updates for edit operations
- Offline-capable draft saving

3.3 Product Engineering Discipline

Our engineering practices directly align with enterprise product development:

- **Trunk-based development** with feature flags for progressive rollout
- **Automated testing pyramid**: Unit tests (xUnit/Jest), integration tests, E2E (Playwright), and dedicated AI output quality tests
- **CI/CD pipeline**: Build → Test → Security scan → Stage deploy → Smoke test → Production deploy
- **Structured PR reviews** with mandatory peer review and automated linting (aligned with our established contribution guidelines)

- **Observability-first**: Application Insights telemetry, custom AI-specific dashboards, SLA monitoring per module

---

4. Module Breakdown & Effort Estimation

> **Note:** Estimates are in person-days and assume full-stack delivery (UI + Backend + AI Agent + RAG + Integration + Testing + Documentation). Ranges reflect complexity uncertainty that would narrow during discovery/design sprints.

4.1 Section A — UI/UX & Workflow Layer Enhancements

A1: SBP (Solution Blueprint) Generation Screens

Sub-Task	Effort (Person-Days)
Discovery & UX design (multi-step wizard, schema editor, export flows)	10–15
Backend — SBP data model, API endpoints, version control service	15–20
AI Agent — Blueprint generation agent, RAG context retrieval, validation	20–25
Frontend — Wizard UI, editable schema sections, diagram viewer, export	20–25
Audit & approval workflow integration	5–10
Testing & QA	10–15
Subtotal	**80–110**

A2: Unit Test Case Generation Screens

Sub-Task	Effort (Person-Days)
Discovery & UX design (developer-centric split-pane interface)	8–10
Backend — Code ingestion pipeline, framework selection service, coverage estimation API	15–20
AI Agent — Unit test generation agent, multi-language support, traceability mapping	20–25
Frontend — Code upload, split-pane editor, coverage preview, framework selector	15–20
CI/CD trigger & repository push integration	8–10
Testing & QA	10–15
Subtotal	**76–100**

A3: Test Script Generation Interface

Sub-Task	Effort (Person-Days)
Discovery & UX design	8–10
Backend — Framework selection engine, environment config service, regression tagging	10–15
AI Agent — Script generation (Selenium/Appium/Hybrid), refactoring suggestions	15–20
Frontend — Script preview with syntax highlighting, cross-browser/device panel, modularization UI	15–20
Migration assistance tooling (legacy → modern)	10–15
Testing & QA	10–13
Subtotal	**68–93**

A4: Test Framework Migration Engine

Sub-Task	Effort (Person-Days)
Discovery & UX design (migration wizard, risk dashboard)	8–10
Backend — Code parsing engine, dependency mapper, framework comparison matrix	20–25
AI Agent — Migration suggestion agent, automated transformation, risk scoring	25–30
Frontend — Analysis dashboard, transformation preview, rollback guidance UI	15–20
Testing & QA (heavy — migration correctness validation)	15–20
Subtotal	**83–105**

4.2 Section B — Engineering Intelligence Modules

B1: HLD/LLD Generation Engine

Sub-Task	Effort (Person-Days)
Discovery & UX design (document structure, diagram generation approach)	10–15
Backend — HLD/LLD data model, traceability service, diagram rendering engine	20–25
AI Agent — Architecture document generation, diagram suggestion, dependency highlighting	25–35
Frontend — Document editor, diagram viewer (component/sequence/data flow), traceability sidebar	20–25
Export (PDF, DOCX, Confluence)	5–8

Sub-Task	Effort (Person-Days)
Testing & QA	10–15
Subtotal	**90–123**

B2: Design Assistance Engine

Sub-Task	Effort (Person-Days)
Discovery & knowledge base curation (architecture patterns, security, performance)	10–15
Backend — Contextual recommendation API, pattern library service	15–20
AI Agent — Design co-pilot agent with contextual awareness, pattern matching	20–25
Frontend — Contextual sidebar/panel, recommendation cards, interactive design chat	15–20
Testing & QA	8–10
Subtotal	**68–90**

B3: Automated Code Generation

Sub-Task	Effort (Person-Days)
Discovery & template framework design	10–15
Backend — Template engine, coding standards enforcement, multi-language support	20–25
AI Agent — Code generation agent, hallucination mitigation, secure coding compliance	30–40
Frontend — Requirements-to-code wizard, code editor with AI suggestions, validation panel	20–25
Testing & QA (quality assurance of generated code — critical)	15–20
Subtotal	**95–125**

B4: Code Reverse Engineering

Sub-Task	Effort (Person-Days)
Discovery & UX design (visualization approaches, output formats)	10–13
Backend — Large codebase parser, API extractor, dependency graph builder	25–35
AI Agent — Architecture map generation, technical debt identification, documentation generation	25–30

Sub-Task	Effort (Person-Days)
Frontend — Graph visualization, interactive dependency explorer, documentation viewer	20–25
Testing & QA	10–15
Subtotal	**90–118**

B5: Advanced Code Impact Analysis

Sub-Task	Effort (Person-Days)
Discovery & UX design	8–10
Backend — Code delta detection, dependency graph traversal, regression scope mapper	20–25
AI Agent — Impact prediction, risk scoring, targeted test suggestion	20–25
Frontend — Impact visualization (tree/graph), risk dashboard, test recommendation panel	15–20
Git integration (PR-triggered analysis)	8–10
Testing & QA	10–15
Subtotal	**81–105**

B6: Automated Code Review

Sub-Task	Effort (Person-Days)
Discovery & UX design	8–10
Backend — Review rule engine, policy configuration, PR workflow integration	15–20
AI Agent — Code analysis agent (performance, security, standards), false positive mitigation	25–30
Frontend — Inline annotation UI, issue severity panel, human-in-the-loop validation	15–20
Git/PR integration (GitHub, Azure DevOps, GitLab)	10–15
Testing & QA	10–15
Subtotal	**83–110**

B7: Code Assessment Integration

Sub-Task	Effort (Person-Days)
Discovery & API mapping for external tools (SonarQube, Veracode, etc.)	8–10

Sub-Task	Effort (Person-Days)
Backend — API integration connectors, metric aggregation engine, correlation service	20–25
AI Agent — Insight generation, cross-tool correlation, actionable recommendation engine	15–20
Frontend — Unified dashboard, drill-down views, story/test correlation panel	15–20
Testing & QA	8–10
Subtotal	**66–85**

4.3 Cross-Cutting Concerns

Item	Effort (Person-Days)
Design system & shared component library buildout	20–30
Agentic Workflow Engine (orchestration framework shared by all modules)	30–40
LLM Router & RAG pipeline enhancements	20–30
RBAC, audit logging, security framework	15–20
CI/CD pipeline setup, environment provisioning	10–15
Observability & monitoring (dashboards, alerting, AI telemetry)	10–15
Documentation & knowledge transfer	10–15
Subtotal	**115–165**

4.4 Effort Summary

Category	Modules	Person-Days (Range)
Section A — Workflow Enhancements	A1, A2, A3, A4	307–408
Section B — Engineering Intelligence	B1, B2, B3, B4, B5, B6, B7	573–756
Cross-Cutting Concerns	Shared infra & frameworks	115–165
Total	**11 Modules + Platform**	**995–1,329**

> **Indicative Duration:** 9–12 months with the proposed team structure (see Section 5), accounting for parallel module development, phased delivery, and iterative refinement.

---

5. Proposed Engagement Model & Team Structure

5.1 Engagement Model

We propose a **Managed Delivery** engagement — a hybrid between fixed-scope and T&M that gives Nomiso cost predictability while preserving agility:

Aspect	Detail
Model	Managed Delivery with quarterly scope checkpoints
Pricing	Monthly retainer based on agreed team composition; scope adjustments handled via change request process at phase boundaries
Governance	Biweekly sprint reviews with Nomiso product team; monthly steering committee with leadership
IP	All code, designs, and documentation produced are Nomiso's IP upon payment
Transparency	Full access to project boards, repositories, CI/CD pipelines, and time tracking

5.2 Proposed Team Composition

Core Team (Full-Time Dedicated)

Role	Count	Responsibilities
Technical Architect / Tech Lead	1	Architecture design, LLM orchestration strategy, code review, cross-module consistency
Senior Backend Engineers	2	Microservices development, API design, data modeling, integration engineering
AI/ML Engineer	1	Agentic workflow development, RAG pipeline, prompt engineering, LLM evaluation
Senior Frontend Engineers	2	React development, design system buildout, complex UI patterns (editors, visualizations, streaming)
Full-Stack Engineer	1	Cross-functional support, integration development, VS Code extension, JIRA plugin
QA Engineer	1	Test strategy, automation (Playwright + API), AI output quality testing
UX Designer	1	Interaction design, wireframes, high-fidelity prototypes, design system governance
Delivery Manager	1 (50%)	Sprint planning, stakeholder communication, risk tracking, dependency management

Total: ~9.5 FTEs

Extended Support (Part-Time / As-Needed)

Role	Allocation	Responsibilities
DevOps Engineer	25%	Infrastructure provisioning, CI/CD pipelines, monitoring setup
Security Consultant	As-needed	Security architecture review, pen testing, compliance alignment
Solutions Architect	Advisory	Monthly architecture reviews, strategic alignment

5.3 Delivery Methodology

Agile Scrum with the following cadence:

Ceremony	Frequency	Participants
Sprint Planning	Biweekly (start of sprint)	Full team + Nomiso PO
Daily Standup	Daily (async for time-zone flexibility)	Dev team
Sprint Review / Demo	Biweekly (end of sprint)	Full team + Nomiso stakeholders
Retrospective	Biweekly	Core team
Backlog Grooming	Weekly	Tech Lead + PO + Architect
Architecture Sync	Biweekly	Architects from both sides
Steering Committee	Monthly	Delivery Manager + Leadership

Definition of Done:

- Code reviewed and merged to main
- Unit tests passing (≥ 80% coverage for new code)
- Integration tests passing
- UI components in Storybook with visual regression tests
- API documentation updated (OpenAPI spec)
- Security scan clean (no critical/high vulnerabilities)
- Validated in staging environment
- Acceptance criteria confirmed by Nomiso PO

5.4 Phased Timeline

```
Phase 0 — Discovery & Foundation          Weeks 1–4
├── Architecture deep-dive with Nomiso engineering team
├── Existing codebase review and integration planning
├── Design system foundation and shared component library
├── Agentic Workflow Engine and LLM Router scaffolding
├── CI/CD and environment setup
└── UX research and module-level wireframes

Phase 1 — Core Enablers + Section A       Weeks 5–20
├── Agentic Workflow Engine (production-ready)
├── RAG pipeline enhancements
├── A1: SBP Generation Screens
├── A2: Unit Test Case Generation Screens
├── A3: Test Script Generation Interface
└── A4: Test Framework Migration Engine

Phase 2 — Engineering Intelligence (Part 1)   Weeks 16–30
├── B1: HLD/LLD Generation Engine
├── B2: Design Assistance Engine
├── B3: Automated Code Generation
└── B5: Advanced Code Impact Analysis
(Phases 1 & 2 overlap by ~4 weeks — parallel streams)
```

```
Phase 3 — Engineering Intelligence (Part 2)   Weeks 26–40
├── B4: Code Reverse Engineering
├── B6: Automated Code Review
├── B7: Code Assessment Integration
├── Cross-module integration testing
└── Performance optimization and security hardening

Phase 4 — Stabilization & Handover            Weeks 38–44
├── End-to-end regression testing
├── Load and performance testing
├── Documentation finalization
├── Knowledge transfer sessions
└── Production deployment support
```

> Phases overlap intentionally — as modules in one phase stabilize, the team pivots capacity to the next phase. This keeps the team fully utilized and compresses the overall timeline.

5.5 Risk Mitigation

Risk	Likelihood	Impact	Mitigation
LLM output quality inconsistency	Medium	High	Multi-layer validation pipeline; human-in-the-loop for critical artifacts; continuous prompt regression testing
Scope creep across 11 modules	High	Medium	Quarterly scope checkpoints; strict change request process; MoSCoW prioritization per module
Integration complexity with existing Intelli-Q	Medium	Medium	Phase 0 deep-dive; dedicated full-stack engineer for integration; continuous integration testing
AI model cost overruns	Medium	Medium	LLM Router with cost-aware model selection; token budgets; caching of repeated patterns
Key personnel dependency	Low	High	Cross-training across modules; documented architecture decisions; pair programming on critical paths
Knowledge base quality affecting AI outputs	Medium	High	RAG quality metrics dashboard; automated freshness scoring; structured feedback loop for retrieval tuning

5.6 IP Protection & Data Privacy

Concern	Approach
Intellectual Property	All deliverables (code, designs, docs, prompts) are Nomiso's property upon payment. Altysys retains no derivative rights.
Data Handling	All development and testing on Nomiso-approved environments. No customer data on developer machines.

Concern	Approach
NDA	Standard mutual NDA covering all project information, architecture details, and business strategy
Access Control	Named individual access with MFA; access revoked within 24 hours of offboarding
LLM Data Privacy	Enterprise-tier LLM deployments (Azure OpenAI) with data processing agreements; no training on customer data; opt-out of telemetry sharing
Compliance	GDPR-aware data handling; SOC 2 alignment for development practices

---

6. Why Altysys

Directly relevant experience:

- **Enterprise AI & Knowledge Platforms** — Built production systems with Azure Cognitive Search, RAG pipelines, and LLM orchestration for enterprise document intelligence (Sound Transit knowledge system — Azure Data Factory, Cognitive Search, .NET APIs)
- **IoT + Cloud at Scale** — Designed and deployed fleet intelligence platform handling 10K+ devices with Azure App Service, Azure SQL, and real-time data pipelines (Draeger)
- **Product Engineering Discipline** — Our internal platform Catalyst demonstrates our approach to modular, testable, enterprise-grade product development: CQRS, event-driven architecture, automated testing (xUnit, Playwright), feature toggles, and structured CI/CD
- **Full-Stack Delivery** — Consistent track record of end-to-end delivery across React frontends, .NET backends, Azure infrastructure, and third-party integrations (Swank, CoachConnect, InvictIQ)
- **AI Market Intelligence** — Active analysis of the AI platform competitive landscape (Articul8, Palantir, Scale AI, Turing) — we understand where Intelli-Q sits in the market and what enterprise buyers expect

What sets us apart for this engagement:

1. **We build products, not projects** — Design system thinking, extensible architecture, and long-term maintainability are built into our approach, not afterthoughts
2. **Azure-native** — Our entire stack runs on Azure. No learning curve on the infrastructure that powers Intelli-Q
3. **Engineering culture aligned with Nomiso's expectations** — Structured PR processes, rigorous code review, automated testing, and documented architecture decisions mirror what you'd expect from a co-innovation partner
4. **AI-practical, not AI-theoretical** — We've shipped LLM-powered features to production. We know the real challenges: prompt brittleness, hallucination mitigation, cost management, and user trust